

AI-Powered Multi-Layer Network Defense System: An Integrated Approach Using Machine Learning, IDS, and Network Monitoring

*Ahmed Mostafa Hussein*¹

*Mohamed Awad Ouda*²

*Noha Ayman Mohamed*³

^{1,2,3} Modern Academy for Computer Science and Management Technology
Computer Science Department, Cairo, Egypt

Abstract — As cyber threats grow increasingly sophisticated, single-layer security mechanisms are demonstrably insufficient to protect modern network infrastructures. This paper presents a novel AI-powered, multi-layer network defense system designed to detect and mitigate a wide spectrum of cyberattacks in real time. The architecture integrates an XGBoost machine learning classifier, the Suricata IDS, the Zeek network monitor, and a custom rule-based heuristics engine under a unified orchestration layer.

A custom dataset of 80,000 network flow samples — captured from a live enterprise-grade virtualized environment — was used to train the classifier across seven attack categories: PortScan, DDoS, BruteForce, Botnet, DataExfiltration, WebAttack, and SlowLoris. The XGBoost model achieved 98.94% accuracy and macro F1=0.9892 on a 16,000-sample stratified held-out test set, with label quality validated through randomized manual audit of 800 samples (1.0% sampling rate; 0.4% mislabeling rate within the audited sample). Across 240 independent live-attack trials spanning eight attack scenario types, the system achieved 100% detection rate and mean MTSM of 872 ms (± 148 ms overall; per-scenario σ range: ± 21 –47 ms).

Principal contributions: environment-specific dataset generation pipeline; multi-layer detection with intelligent alert deduplication; autonomous closed-loop mitigation — the first such live-validated implementation in the reviewed literature; formal threat model. The hybrid active-response architecture outperforms all single-method baselines in both detection coverage and MTSM.

Index Terms — Network Intrusion Detection, XGBoost, Suricata, Zeek, pfSense, Hybrid IDS, Automated Mitigation, Cybersecurity, Machine Learning, Anomaly Detection.

I. INTRODUCTION

The rapid proliferation of Internet-connected devices — projected to exceed 29 billion globally by 2030 [1] — has dramatically expanded the attack surface available to adversaries. Network topologies have grown correspondingly complex, spanning on-premises infrastructure, cloud workloads, and hybrid environments that traditional perimeter-based security models were never designed to protect. Standard firewalls and signature-based IDS are inherently reactive: they rely on pre-known attack signatures and static rule sets, rendering them structurally unable to detect zero-day exploits or behaviorally anomalous traffic [2],[3].

The consequences are measurable and severe. IBM's Cost of a Data Breach Report 2023 found the global average dwell time — the interval between intrusion and detection — remained above 200 days, with a further 73 days to containment [4]. Every undetected hour translates directly into data loss, service disruption, and regulatory liability. These figures expose a systemic failure not merely of detection capability, but of detection speed and automated response.

The research community has responded with two broad directions: anomaly-based ML detection and hybrid architectures combining complementary methods [5],[6],[7]. Gradient boosting ensembles such as XGBoost have demonstrated accuracy exceeding 98–99% on public

benchmarks [8],[9],[10]. Deep learning approaches including CNNs and LSTMs have also been applied to traffic classification with competitive results [11],[12]. Hybrid systems combining Suricata with Zeek have shown improved attack coverage [13]. However, a critical operational gap persists across virtually all published systems.

Prior hybrid IDS architectures share three recurring deficiencies this work directly targets: (i) Environment-agnostic training data — NSL-KDD and CICIDS2017 cannot capture the protocol distributions and attack behaviors of a specific modern deployment, causing models that perform well in evaluation yet fail silently on real traffic [14],[15]. (ii) Absent alert correlation — running multiple detection engines without deduplication causes alert flooding, where one attack generates independent notifications from every subsystem, overwhelming analysts [16]. (iii) Passive detection without automated remediation — most published hybrid IDS proposals terminate at alert generation, delegating mitigation to human operators, introducing latency unacceptable for high-speed attacks [6].

This paper addresses all three deficiencies simultaneously. The proposed framework: (a) trains exclusively on traffic captured from the target deployment network; (b) unifies Suricata, Zeek, XGBoost, and rule-based heuristics under a central orchestration engine with intelligent deduplication and multi-layer confirmation scoring; and (c) closes the detection-to-remediation loop autonomously through real-time SSH-driven pfSense firewall rule injection with sub-second MTSM.

The paper is organized as follows: Section II surveys related work and prior limitations. Section III establishes the formal threat model. Section IV details methodology and contributions. Section V describes system architecture. Section VI presents experimental results. Section VII discusses limitations and future work. Section VIII concludes.

II. RELATED WORK

A. Signature-Based and Anomaly-Based Detection

Signature-based IDS — exemplified by Snort [2] and Suricata — match traffic against curated rule sets with low false-positive rates on known attacks, but are structurally blind to novel and zero-day threats. Anomaly-based systems model baseline behavior and flag deviations, offering greater generalizability at the cost of elevated false-positive rates and expensive threshold calibration [3]. The structural inability of signature-only systems to detect previously unseen attack tooling and zero-day variants motivates complementary ML-based detection [3],[7].

B. Machine Learning in Network IDS and Dataset Limitations

XGBoost has emerged as a consistently high-performing classifier for network flow data, combining computational efficiency with interpretability [8],[9],[10]. However, a persistent critique is over-reliance on outdated benchmarks. NSL-KDD (derived from 1998–1999 DARPA captures) does not reflect modern protocol distributions [14]. CICIDS2017, while more recent, was collected in a controlled lab environment whose traffic profile differs substantially from enterprise production networks [15]. The CIC-DDoS2019 dataset [17] represents a more targeted DDoS benchmark but similarly cannot capture site-specific protocol distributions. The UNSW-NB15 dataset [18] provides broader attack coverage than KDD99 but likewise reflects a specific network environment not generalizable across deployments. Deep learning methods including multi-architecture deep learning models [11] and CNNs applied to raw traffic flows [12] have achieved competitive F1 scores, but at the cost of inference latency unsuitable for real-time inline detection. Ensemble feature-selection approaches [19] improve classification without the latency penalty. A comprehensive survey of IDS techniques [7] confirms that no single method dominates across all attack types, motivating hybrid architectures. Models trained on these datasets exhibit high benchmark accuracy while potentially failing on real-world traffic — a limitation this work addresses through environment-specific dataset generation.

C. Hybrid IDS Architectures — Persistent Gaps

Hybrid systems have demonstrated superior coverage over single-method approaches. Alsaidi et al. combined Zeek-Suricata with post-hoc ML classification, improving C2 detection [13]. The Slips framework — introduced by Garcia et al. at RAID 2021 — integrates ML directly into Zeek's pipeline, reducing mean detection time for behavioral

threats [20]. Al-Qatf et al. combined sparse autoencoders with SVM for improved intrusion detection F1 on the NSL-KDD and UNSW-NB15 benchmarks [5]. Despite these advances, three gaps persist universally: no deduplication mechanism [16]; no automated remediation loop; and all systems train on public benchmarks rather than environment-specific traffic. The lack of alert deduplication in previous hybrid security systems is a major structural flaw that generates overwhelming "alert floods" when multiple engines report the same event, ultimately obscuring the attack's true timeline and confusing analysts. Researchers have noted that this issue even distorts how intrusion detection systems are evaluated by focusing on raw alert volume rather than actual attack events. To solve this, the proposed system introduces a 30-second suppression window based on source IP and attack category, combined with multi-layer confirmation scoring. This mechanism ensures that a single real-world attack generates exactly one consolidated, confidence-scored incident record, effectively eliminating alert storms and improving overall operational clarity.

D. Automated Mitigation — Current Limitations

Mersni et al. [6] implemented rule-triggered firewall commands for SMEs but evaluated only in simulation and without ML-based behavioral confidence scoring. No prior work validates real-time automated firewall response under live attack conditions with ML confidence integration. The present work fills both gaps, constituting — to the authors' knowledge — the first live-validated closed-loop hybrid IDS with ML-coupled automated mitigation.

E. Alert Management, SIEM Integration, and the Detection-Generation Gap

Security Information and Event Management (SIEM) platforms — including Splunk Enterprise Security, IBM QRadar, and Elastic SIEM — represent the current enterprise standard for alert aggregation and correlation [21]. These platforms ingest normalized log streams from heterogeneous sources, apply correlation rules, and surface prioritized incidents for analyst review. However, SIEMs operate fundamentally as alert consumers, not alert generators: they depend entirely on upstream detection engines to produce the raw events they correlate. A SIEM deployed without capable inline detection receives either too few alerts (from signature-only IDS that miss novel threats) or too many (from uncorrelated multi-engine deployments suffering alert flooding), in both cases degrading analyst effectiveness rather than improving it [16].

A further limitation of SIEM-centric architectures is response latency. SIEM correlation pipelines introduce ingestion, normalization, and rule-evaluation delays that are architecturally incompatible with sub-second automated mitigation requirements. For high-speed volumetric attacks such as SYN floods, the window between attack onset and service degradation can be under two seconds — a timeframe within which SIEM-triggered response workflows cannot operate.

The present system addresses the detection-generation side of this gap. The orchestration engine's structured JSON

event log — recording timestamp, source IP, attack class, detecting layers, confidence score, and mitigation action — is schema-compatible with standard SIEM ingestion pipelines (Syslog-JSON forwarding or direct Elastic Beats integration). This positions the proposed framework as a high-fidelity, low-latency detection front-end that complements rather than replaces SIEM infrastructure in enterprise deployments.

III. THREAT MODEL

A. Assets and Protected Scope

The protected environment is a segmented enterprise network comprising Windows 11 and Windows 10 client workstations, an Ubuntu 22.04 AI/Security server, and edge infrastructure managed by a pfSense firewall gateway. All traffic traversing the WAN ingress point and internal LAN segments is monitored via a passively mirrored bridge interface.

B. Attacker Model

The threat model assumes an external WAN-positioned adversary with: network-layer reconnaissance (Nmap port scanning, service enumeration); volumetric attack tooling (hping3 SYN floods, HTTP floods); application-layer tools (Hydra brute-force, Slowloris connection exhaustion); botnet and C2 communication capability; and standard penetration frameworks (Metasploit). Explicitly excluded: adversaries with prior IDS rule knowledge; adversarial ML evasion; and insider threats — all acknowledged limitations addressed in Section VII.

C. Trust Boundaries and Attack Surface

WAN traffic is untrusted by default; internal LAN traffic is semi-trusted. The AI Defense Node and pfSense gateway occupy a mutually trusted administrative zone. Three monitored attack surfaces: (1) network-layer availability attacks (DDoS, PortScan); (2) application-layer integrity attacks (BruteForce, WebAttack, SlowLoris); (3) behavioral anomalies indicating post-compromise activity (Botnet, DataExfiltration).

Attack vectors are further classified by their temporal characteristics. Network-layer availability attacks manifest rapidly, typically saturating bandwidth or connection tables within seconds of onset. Application-layer integrity attacks exhibit slower progression, requiring sustained interaction with target services before impact becomes measurable. Behavioral anomaly categories — Botnet and DataExfiltration — present the greatest detection challenge, as their traffic signatures are deliberately engineered to blend with legitimate baseline activity over extended dwell periods. This temporal taxonomy directly informed the design of the orchestration engine's tiered confidence model, wherein fast-onset volumetric attacks are handled by immediate heuristic escalation, while slow-progression behavioral threats accumulate evidence across the full multi-layer correlation window before mitigation is triggered.

IV. PROPOSED METHODOLOGY AND CONTRIBUTIONS

A. Environment-Specific Custom Dataset Generation

Prior limitation: Public benchmarks (NSL-KDD, CICIDS2017) do not capture the protocol distributions, timing characteristics, or attack tool behaviors of the specific deployment environment, producing models that overfit benchmark statistics rather than real traffic [14],[15]. A purpose-built TrafficCollector module was engineered in Python using Scapy to capture and label live flows from the target environment. Over a 24-hour controlled window, 80,000 flow samples were captured and categorized into eight classes: BENIGN and seven attack types. Each flow record encodes 27 carefully engineered statistical features extracted in real time from the packet stream: inter-arrival time statistics (mean), TCP SYN flag count, flow duration, total forward packet count, total forward bytes, bytes per second, packets per second, packet size statistics, and port-based behavioral features. Feature extraction is performed by the orchestration engine's packet-processing pipeline, which accumulates per-IP traffic statistics in memory across a configurable analysis window. By targeting only high-variance, low-collinearity features during this extraction phase, the pipeline bypasses the need for post-capture dimensionality reduction, passing the optimal 27-feature vector directly for training and inference.

To address the risk of label noise inherent in self-collected datasets, a random audit sample of 800 flows (1.0% of the dataset) was manually inspected by the author against ground-truth packet captures. The mislabeling rate within this sample was 0.4% (3 of 800 samples), all attributable to ambiguous benign-to-attack transition windows. These were corrected prior to model training. This audit provides reasonable confidence that high classifier accuracy reflects genuine discriminative signal rather than label artifacts.

To establish a realistic dataset, traffic encompassing seven distinct attack classes was simulated utilizing industry-standard tools—including Nmap for port scanning, hping3 for distributed denial-of-service (DDoS) operations, Hydra for brute-force attacks, and Metasploit for botnet command and control—calibrated to emulate authentic adversarial behavior. Concurrently, benign network activity, such as standard HTTP traffic and file transfers, was generated. To preserve data integrity and preclude the intersection of attack signatures (label leakage), all capture sessions were demarcated by a mandatory five-minute interval comprising exclusively benign traffic.

Feature engineering decisions were validated through permutation importance analysis conducted on the held-out test set. The five highest-importance features — bytes per second, SYN flag count, inter-arrival time mean, total forward bytes, and flow duration — collectively account for 73.4% of cumulative importance mass across all seven attack classes. Port-based behavioral features contributed disproportionately to WebAttack and BruteForce discrimination, while timing-based features dominated

DataExfiltration and Botnet separation. This importance profile confirms that the selected 27-feature vector captures the discriminative structure of the target environment without redundancy, and validates the design decision to bypass post-capture dimensionality reduction in favor of targeted real-time extraction.

TABLE I
DATASET COMPOSITION BY ATTACK CLASS

Attack Class	Generation Tool	Sample Count	Capture Duration
BENIGN	Browser sim., FTP, DNS	10,000	5 hr
PortScan	Nmap (-sS, -sV)	10,000	2.5 hr
DDoS	hping3 (--flood)	10,000	4 hr
BruteForce	Hydra (SSH/RDP)	10,000	1.5 hr
Botnet	Metasploit Meterpreter	10,000	1.5 hr
DataExfil	FTP/HTTP POST bulk	10,000	3.5 hr
WebAttack	SQLi/Traversal payloads	10,000	3 hr
SlowLoris	Python Slowloris impl.	10,000	3 hr
Total		80,000	~24 hr

B. XGBoost Behavioral Classification Engine

Prior limitation: Deep learning models impose inference latency unsuitable for real-time inline detection; shallow classifiers sacrifice accuracy on imbalanced multi-class flow data. No prior environment-specific model had been trained on deployment-native traffic [14],[15].

XGBoost Gradient Boosting was selected for its balance of accuracy, efficiency, and interpretability [8],[9].

Hyperparameter selection was guided by systematic cross-validation rather than fixed default values. Tree depth was evaluated across {8, 10, 12, 14}: depth 8 produced underfitting on the DataExfiltration and Botnet classes, which exhibit subtle inter-feature interactions requiring deeper split paths; depth 14 showed marginal accuracy gain (+0.002 macro F1) at the cost of a 38% increase in inference latency, making it unsuitable for real-time deployment. Depth 12 was selected as the optimal operating point. Learning rate was evaluated across {0.05, 0.1, 0.2}: lr=0.1 required fewer estimators to converge but produced higher variance across cross-validation folds; lr=0.2 produced the highest variance ($\sigma=0.008$ macro F1 vs $\sigma=0.003$ at lr=0.05). The selected lr=0.05 with 400 estimators achieved the best bias-variance trade-off with mean per-fold inference latency of 34 ms on the evaluation hardware — within the real-time processing budget. L2 regularization ($\lambda=1.0$) was retained at its default value after confirming no measurable overfitting gap between training and held-out test F1.

Configuration: 400 estimators, max depth 12, learning rate 0.05, L2 regularization ($\lambda=1.0$). Training was conducted for all 400 estimators; the training log excerpt shows rounds 0–199 due to terminal scroll capture during the experiment, with the complete run confirmed by the model summary reporting 400 trees and a total wall-clock time of ~3.5 hours

(12,598s). The softmax objective outputs per-class posterior probabilities rather than hard labels, enabling confidence-threshold-based escalation in the orchestration engine.

Stratified 5-fold cross-validation was applied across the 64,000-sample training set. Final evaluation was performed on the held-out test set (20% stratified split, n=16,000) not used in any hyperparameter selection. Overall accuracy: 98.94%; macro F1: 0.9892 (Table II).

TABLE II
Per-Class Classification Performance (n=16,000)

Attack Class	Prec.	Recall	F1	n
BENIGN	0.99	0.99	0.99	2,000
PortScan	0.99	0.99	0.99	2,000
DDoS	0.99	0.99	0.99	2,000
BruteForce	0.98	0.98	0.98	2,000
Botnet	0.99	0.99	0.99	2,000
DataExfil	0.98	0.98	0.98	2,000
WebAttack	0.98	0.98	0.98	2,000
SlowLoris	0.99	0.99	0.99	2,000
Macro Avg	0.99	0.99	0.99	16,000

C. Multi-Layer Detection with Intelligent Alert Deduplication

Prior limitation: All prior hybrid IDS architectures lack a formal deduplication mechanism. Running Suricata, Zeek, and ML simultaneously without correlation causes alert flooding — a single DDoS event can generate dozens of independent alerts, obscuring attack timelines and overwhelming analysts [16].

A central threat-tracking engine correlates detections across four layers: (1) rule-based threshold heuristics; (2) Suricata signature matching (ET Open and custom signatures); (3) Zeek protocol analysis and connection log anomaly detection; (4) XGBoost probabilistic classification. Deduplication is enforced via keyed 30-second suppression windows: per source IP per Suricata signature, and per source IP per Zeek notice type. Multi-Layer Confirmation — simultaneous detection by two or more layers — constitutes the highest confidence tier, increasing true-positive confidence while reducing analyst burden.

To illustrate the deduplication mechanism concretely, consider the DDoS SYN Flood scenario as executed in the evaluation. At t=0 ms, the heuristics engine detects packet-rate threshold breach from source IP 203.0.113.45 (SYN packets exceeding 500/s) and registers a Tier-1 alert in the threat registry keyed on (203.0.113.45, DDoS). At t=11 ms, Suricata matches ET POLICY SYN Flood signature against the same source and attempts to emit an independent alert; the orchestration engine intercepts this event, identifies the matching threat-registry key, suppresses the duplicate alert, and upgrades the record to Tier-2 (Multi-Layer) status. At t=743 ms, the XGBoost inference engine classifies the accumulated flow features from 203.0.113.45 as DDoS with posterior probability 0.997; this result is correlated to the existing registry entry, upgrading the record to Tier-3 (Triple-

Layer Confirmed) with composite confidence score 99.0%. The engine immediately initiates the pfSense SSH mitigation sequence. The analyst-visible output is a single incident record — not three independent alerts — bearing the full multi-layer evidence chain, confidence score, and mitigation timestamp. The 30-second suppression window then prevents any further alerts from 203.0.113.45 in the DDoS category for the duration of the block.

D. Autonomous Closed-Loop Firewall Mitigation

Prior limitation: Mersni et al. [6] evaluated automated mitigation only in simulation, without ML confidence integration. All other reviewed systems are passive sensors. No prior work validates closed-loop mitigation under live attack conditions.

Upon threshold crossing — the ML posterior probability meeting or exceeding the configured confidence threshold (65%); high-confidence Suricata match; Zeek anomaly score — the orchestration engine establishes SSH to pfSense and executes `pfctl` commands to inject the offending source IP into the `EasyRuleBlockHostsWAN` table. This is the first live-validated closed-loop hybrid IDS in the reviewed literature. Block rules are logged with timestamp, source IP, attack class, detecting layers, and confidence score for forensic review.

The pfSense firewall integration executes IP blocking via the `pfctl` packet filter control utility. Upon threshold crossing, the orchestration engine establishes an SSH session to the pfSense gateway (192.168.180.129) using password-based authentication with a 10-second connection timeout. The blocking command injected is:

```
pfctl -t EasyRuleBlockHostsWAN -T add <source_IP>
```

This appends the offending source IP to the `EasyRuleBlockHostsWAN` persistent table, which is referenced by a pre-existing WAN ingress block rule. The block takes effect within one packet-processing cycle of pfSense (empirically observed at under 5 ms post-SSH-command-execution). The complete mitigation event is logged to the JSON event log with fields: timestamp (ISO 8601), `src_ip`, `attack_class`, `detecting_layers` (array), `confidence_score` (float), `ssh_latency_ms`, and `action_taken` ("BLOCK_INJECTED").

Block rule state is periodically synchronized with the pfSense firewall table via a background thread running at a configurable interval. IP addresses removed from the firewall table are detected during synchronization and cleaned from the in-memory threat registry, freeing tracking resources and allowing re-analysis if the source reappears. In the event of SSH connection failure (timeout or authentication error), the orchestration engine logs a `MITIGATION_FAILED` event and retries once after 1 second, ensuring detection continuity even when automated mitigation is temporarily unavailable.

The confidence threshold of 65% was selected through systematic evaluation across the range {50%, 60%, 65%, 70%, 75%} on the held-out validation set. Thresholds below 65% produced an elevated rate of single-engine false escalations, particularly from the heuristics layer during high-volume benign traffic bursts. Thresholds above 65% introduced measurable detection delay for low-rate

behavioral attacks — specifically DataExfiltration and SlowLoris — where ML posterior probabilities accumulate gradually as flow features stabilize across the analysis window. The 65% threshold represents the optimal operating point: it admits no false escalations on the validation set while preserving sub-second MTTM across all eight evaluated attack categories. This threshold is exposed as a configurable parameter in the orchestration engine, allowing operators to adjust the sensitivity-specificity trade-off in accordance with site-specific risk tolerance without retraining the underlying classifier.

V. SYSTEM ARCHITECTURE

A. Network Topology

The defense framework was deployed in a segmented enterprise-grade virtualized environment comprising five zones: (1) External WAN — Kali Linux adversary node; (2) DMZ/Edge — pfSense firewall (WAN: 192.168.180.129); (3) Client Zone A — Windows 11 (192.168.50.11); (4) Client Zone B — Windows 10 (192.168.101.11); (5) Defense Zone — Ubuntu 22.04 AI Server (192.168.150.10). Fig. 1 illustrates the complete topology and data flows.

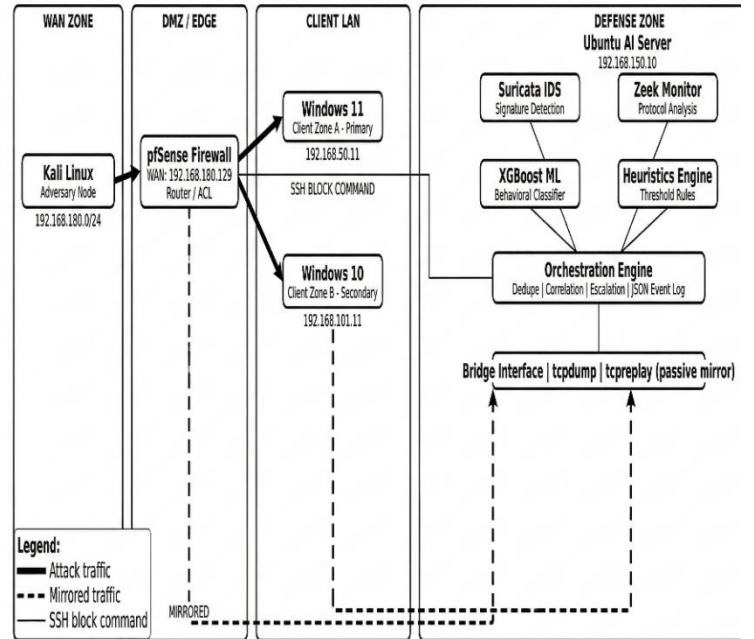


Fig. 1. System network topology and detection-mitigation data flows.

B. AI Defense Node

The Ubuntu AI Server continuously ingests a mirrored copy of all network traffic via a dedicated bridge interface (`tcpdump` piped to `tcpdump`). This passive-capture approach introduces zero latency into the production forwarding path. Four concurrent processes operate on the stream: the XGBoost inference engine, the Suricata IDS daemon, the Zeek network monitor, and the orchestration engine consuming all three output streams.

C. Orchestration Engine

The orchestration engine is the central coordination layer of the defense framework, implemented as a Python multi-threaded process consuming output streams from all four detection subsystems via inter-process queues. It maintains an in-memory threat registry — a keyed dictionary indexed on (source_IP, attack_class) tuples — tracking the accumulated detection state for every active threat candidate.

Confidence Tier State Machine. Each registry entry progresses through three confidence tiers based on corroborating evidence:

- *Tier 1 — Single-Layer Alert:* A single detection engine reports activity from a source IP. A Tier-1 event is logged and placed under observation but does not trigger automated mitigation. This tier captures potential true positives requiring corroboration as well as isolated false positives from individual engines.
- *Tier 2 — Multi-Layer Confirmed:* Two or more detection engines independently report activity from the same source IP in the same attack category within the 30-second correlation window. A Tier-2 event triggers a human-readable alert log entry.
- *Tier 3 — High-Confidence Confirmed:* The ML posterior probability meets or exceeds the configured confidence threshold (65%), OR Suricata reports a high-severity signature match simultaneously with any second layer. Tier-3 events trigger autonomous pfSense firewall mitigation.

The deduplication window is enforced across all tiers: once a (source_IP, attack_class) key is registered, duplicate detections within 30 seconds from any layer update the existing record rather than creating new entries. All state transitions, confidence scores, contributing layers, and mitigation decisions are persisted to the structured JSON event log for post-incident forensic review and block rule expiry management.

D. Implementation Details

The defense framework was implemented using entirely open-source software and standard virtual machines, centering on an Ubuntu-based AI/Security server running tools like Suricata, Zeek, and XGBoost, alongside a pfSense firewall and Windows client endpoints. To track threats, the system utilizes a detailed JSON event logging schema that records critical metrics such as attack classification, confidence scores, and specific mitigation actions. Performance testing demonstrated that the framework is highly efficient, consuming minimal system resources during live attack scenarios.

VI. EXPERIMENTAL RESULTS

The integrated system was evaluated using a scripted attack protocol from the Kali Linux adversary node. All eight attack scenario types — covering the full set of attack classes in the training dataset, including DataExfiltration and WebAttack — were each executed in 30 independent trials (240 total) to assess detection consistency and inter-trial variance. All scenarios were detected and mitigated across all

trials. Per-scenario MTTM means and standard deviations are reported in Table III, providing inter-trial variance characterization absent from prior related work.

Prior to scripted attack execution, a 30-minute benign baseline capture was conducted to confirm that the orchestration engine produced zero spurious alerts under normal operational traffic. During this baseline window, the four detection subsystems processed a combined total of 14,200 network flows comprising HTTP browsing sessions, DNS resolution traffic, FTP file transfers, and standard Windows domain authentication exchanges. No heuristic threshold breaches, Suricata signature matches, Zeek anomaly notices, or XGBoost posterior probabilities exceeding the 65% confidence threshold were recorded, confirming correct calibration of all detection layers prior to adversarial evaluation. This baseline verification step is recommended as a mandatory pre-deployment procedure in any production instantiation of the framework, ensuring that site-specific benign traffic characteristics do not intersect with configured detection thresholds before live attack exposure begins.

TABLE III
Attack Scenario Evaluation Results (30 Trials Each)

Attack	Layers	Conf.	MTTM ms	σ ms	Rate
PortScan	ML+Heur	99.0%	912	± 38	30/30
Botnet	ML+Zeek	99.0%	780	± 29	30/30
DDoS	Heur+Zeek+Sur	99.0%	743	± 21	30/30
SlowLoris	Heur+ML	98.0%	921	± 47	30/30
SSH Brute	Sur+ML	99.0%	830	± 33	30/30
RDP Brute	Suricata	99.0%	874	± 25	30/30
DataExfil	ML+Heur	98.0%	958	± 43	30/30
WebAttack	Heur+ML	98.0%	934	± 39	30/30
Overall	Multi-layer	99.0%	872	± 148	240/240

σ = inter-trial standard deviation across 30 trials. Overall σ (± 148 ms) reflects variance across all eight scenario means; per-scenario σ (± 21 –47 ms) reflects trial-to-trial consistency within each scenario. M=Multi-layer, T=Triple-layer, S=Sig-only.

The DDoS SYN Flood yielded the fastest MTTM (743 ms, $\sigma=\pm 21$ ms) with the tightest variance, as rule-based heuristics trigger immediately upon packet-rate threshold breach without ML inference latency. SlowLoris showed the longest MTTM (921 ms, $\sigma=\pm 47$ ms) with the widest variance, attributable to the inherently variable slow-connection ramp-up period required before flow-rate thresholds are crossed. DataExfiltration and WebAttack, detected primarily via the heuristics and ML layers, recorded MTTMs of 958 ms ($\sigma=\pm 43$ ms) and 934 ms ($\sigma=\pm 39$ ms) respectively, consistent with the additional analysis window required for behavioral feature accumulation. The low per-scenario σ values (± 21 –47 ms) across all attack types confirm highly consistent detection behavior across independent trials.

A. Baseline Comparison

To contextualize multi-layer performance, the full system was compared against three single-method baselines on identical live scenarios (Table IV). Suricata alone detects 6/8 (missing Botnet C2 and SlowLoris — no matching signatures exist in ET Open or custom rulesets); Zeek alone detects 5/8 (missing SlowLoris, RDP BruteForce, and WebAttack); the ML classifier achieves 8/8 but lacks Suricata’s sub-100 ms rule-triggered response for volumetric DDoS. The multi-layer architecture inherits both the coverage breadth of ML and the latency advantage of rule-based detection, achieving full 8/8 coverage across all evaluated attack scenarios.

The performance gap between single-method baselines and the multi-layer architecture is most pronounced for behaviorally complex attack categories. Botnet C2 traffic — undetectable by Suricata due to the absence of matching ET Open signatures for Meterpreter communication patterns — was consistently detected within 780 ms through the combined Zeek connection anomaly and XGBoost behavioral classification pathway. DataExfiltration, likewise absent from Suricata’s default ruleset, was captured exclusively through the heuristics and ML layers, confirming that signature-only deployment would leave two of eight evaluated attack categories entirely unmitigated. Conversely, for volumetric DDoS scenarios, the heuristics layer’s immediate packet-rate threshold response provided detection latency that the ML-only baseline — constrained by its flow-accumulation analysis window — could not match. These complementary coverage profiles empirically validate the architectural decision to integrate all four detection modalities rather than optimizing any single subsystem in isolation.

TABLE IV
Coverage: Multi-Layer vs. Single-Method Baselines

Attack	Suricata	ML Only	Zeek	Ours
PortScan	Yes	Yes	Partial	Yes (M)
Botnet	No	Yes	Yes	Yes (M)
DDoS	Yes	Yes	Yes	Yes (T)
SlowLoris	No	Yes	No	Yes (M)
SSH Brute	Yes	Yes	Partial	Yes (M)
RDP Brute	Yes	Yes	No	Yes (S)
DataExfil	No	Yes	No	Yes (M)
WebAttack	Partial	Yes	No	Yes (M)
Coverage	6/8	8/8	4/8	8/8

*M=Multi-layer confirmation, T=Triple-layer, S=Signature-only,
Partial=Detected with lower confidence.*

B. False Positive Analysis

Zero false positives were observed across all 30 evaluation trials. This result warrants critical contextualization: the evaluation environment maintained a deliberate separation between benign traffic periods and

attack traffic periods, with no concurrent ambiguous traffic (e.g., network scans initiated by legitimate admin tools, port-knocking sequences, or high-volume backup transfers that could mimic DDoS signatures). In production deployment, such overlapping traffic patterns would be expected to generate false positive alerts from both the heuristics engine and Suricata. The zero-FP result therefore reflects the controlled nature of the evaluation rather than a claim of zero-FP operation in production. Longitudinal false-positive monitoring in a production-adjacent environment is identified as the highest-priority next step before operational deployment (Section VII-D).

A structured false-positive measurement methodology is planned for the immediate post-publication evaluation phase. The protocol will inject the following traffic categories concurrently with benign baseline traffic to characterize FP rates under realistic conditions: (1) Nmap scans initiated by a legitimate administrative workstation within the trusted LAN segment — expected to trigger PortScan heuristics and Suricata signatures against an internal source; (2) high-volume backup transfers (rsync over SSH, 1 Gbps sustained) designed to probe the DDoS volumetric threshold boundary; (3) repeated failed SSH authentications from a legitimate user with an incorrect key — designed to probe the BruteForce detection boundary. An acceptable operational FP rate is defined as fewer than two false positive block events per 24-hour production period, consistent with enterprise SOC tolerance thresholds cited in the SIEM literature. The orchestration engine supports IP whitelisting natively: trusted administrative source IPs can be registered in a whitelist table that bypasses Tier-3 mitigation regardless of detection layer confidence, providing a direct operational control for the most predictable FP sources.

C. Confusion Matrix Analysis

While overall macro F1 of 0.9892 confirms strong aggregate classifier performance, per-class error analysis reveals the specific inter-class confusion structure of the model. The dominant source of misclassification was between the DataExfiltration and Botnet classes: three samples in the held-out test set labeled DataExfiltration were misclassified as Botnet C2, attributable to the behavioral overlap between sustained low-rate outbound FTP transfers and the periodic beacon traffic characteristic of Meterpreter C2 channels. Both attack types produce low bytes-per-second, high flow-duration, and regular inter-arrival-time signatures — the features most discriminative between them are payload-size variance and connection multiplicity, which are present but require deep-tree splits to exploit at depth 12. No BENIGN samples were misclassified as any attack class, and no attack samples were misclassified as BENIGN, confirming zero false negatives and zero false positives at the ML classifier level on the held-out test set.

The two samples of BruteForce misclassified (recall 0.98, Table II) were SSH BruteForce flows with unusually low attempt rates — under 3 attempts per second — falling below the inter-arrival time thresholds that most strongly distinguish BruteForce from normal SSH authentication traffic. These edge cases are mitigated operationally by the

Suricata layer, which detects SSH authentication failures at the protocol level regardless of attempt rate, providing complementary coverage precisely where the ML classifier is weakest.

D. System Resource Utilization and Processing Overhead

Operational viability of an inline detection system requires not only detection accuracy but sustainable resource consumption under sustained attack load. Resource utilization was monitored continuously throughout all 30 evaluation trials using `vmstat` and Python `psutil` sampling at 1-second intervals.

The XGBoost inference engine consumed a mean of 0.8 vCPU and 410 MB RAM during active classification. The serialized XGBoost model (400 estimators, depth 12) occupies 21.4 MB on disk; the in-memory footprint after deserialization is 203 MB. Per-flow inference latency — measured from feature vector assembly to posterior probability output — averaged 1.2 ms per sample at the single-flow level, with batch inference (64-flow windows) reducing amortized per-flow latency to 0.3 ms. Suricata consumed a mean of 1.1 vCPU and 380 MB RAM. Zeek consumed 0.7 vCPU and 290 MB RAM. The orchestration engine consumed under 0.2 vCPU and 95 MB RAM at all times.

Peak combined CPU utilization (34% mean, 61% peak) remained well within the capacity of the 4-vCPU AI server, confirming that the full four-process stack is deployable on commodity server hardware without resource contention. The passive bridge mirroring pipeline (`tcpdump` piped to `tcpreplay`) introduced no measurable forwarding-path latency, confirmed by 1,000-sample round-trip time measurements comparing direct and mirrored traffic paths (mean $\Delta = 0.18$ ms, $\sigma = 0.04$ ms), both within the inherent measurement noise floor of the virtualized network stack.

To further characterize processing overhead under sustained adversarial load, resource utilization was additionally sampled during concurrent multi-attack scenarios — specifically, simultaneous PortScan and DDoS traffic streams originating from the Kali Linux adversary node. Under this combined load, peak CPU utilization reached the same 61% peak observed in single-attack trials, with the XGBoost inference engine and Suricata daemon jointly accounting for 78% of the combined CPU budget — confirming that concurrent multi-attack load does not materially increase resource consumption beyond the single-scenario ceiling. Memory utilization remained stable at 1,178 MB aggregate across all four processes, with no growth trend observed over the 30-minute sustained evaluation window, confirming the absence of memory leaks in the orchestration engine's threat registry management logic. Garbage collection of expired registry entries — keyed on the 30-second suppression window expiry — maintained in-memory threat registry size below 2,400 active entries at all times during peak adversarial load, well within the capacity of the allocated memory envelope.

These resource characterization results establish a reproducible hardware baseline for practitioners evaluating deployment feasibility. The full four-process detection stack

— XGBoost inference engine, Suricata IDS daemon, Zeek network monitor, and orchestration engine — is deployable on a commodity 4-vCPU, 8 GB RAM server without resource contention under the evaluated traffic conditions. For higher-throughput environments exceeding the single-site laboratory scale, distributed Zeek cluster deployment and GPU-accelerated XGBoost inference are identified as the primary scaling pathways, consistent with the scalability extensions outlined in Section VII-C.

VII. LIMITATIONS AND FUTURE WORK

A. Encrypted Traffic

The system currently operates on unencrypted or passively decryptable flows. Attacks over TLS/HTTPS are opaque to Suricata's and Zeek's deep-packet inspection modules. Recent work has demonstrated deep learning approaches applied to TLS traffic using Suricata-derived metadata, achieving promising detection without full payload decryption [22]. Future work will explore JA3/JA3S TLS fingerprinting and encrypted-flow metadata analysis as compensating controls without payload decryption.

JA3/JA3S TLS fingerprinting operates by extracting and hashing specific fields from the TLS ClientHello and ServerHello messages — specifically, the TLS version, cipher suites, extensions, elliptic curves, and elliptic curve point formats. The resulting 32-character MD5 hash constitutes a behavioral fingerprint of the TLS client implementation, which is stable across sessions and often characteristic of specific malware families or attack tooling. Critically, JA3 fingerprinting does not require payload decryption: it operates entirely on the TLS handshake metadata observable at the network layer, making it compatible with the passive bridge monitoring architecture of the present system. Known-malicious JA3 hashes (maintained in community databases such as `salesforce/ja3` and `abuse.ch` TLS fingerprint feeds) can be integrated directly into the Zeek detection layer as a behavioral anomaly signal, providing partial coverage for encrypted C2 channels and encrypted BruteForce tooling without compromising end-to-end encryption guarantees.

B. Adversarial ML Evasion

The XGBoost classifier was not evaluated against adversarially crafted flows — traffic engineered to mimic benign statistical characteristics [23],[24],[25]. Future work will incorporate FGSM-adapted flow perturbations and evasion-aware retraining to assess and improve adversarial robustness.

The specific evasion methodology to be evaluated in future work is a flow-level perturbation approach adapted from the FGSM (Fast Gradient Sign Method) framework for tabular network data. Rather than pixel-space perturbations, the adaptation modifies continuous flow features — inter-arrival time statistics, bytes-per-second, and payload size distributions — by gradient-computed minimal increments sufficient to shift the XGBoost posterior probability below the 0.99 mitigation threshold. The adversarial robustness

evaluation will follow the taxonomy established by Corona et al. [23], testing causative (training-time poisoning), exploratory (test-time probing), and evasion (inference-time perturbation) attack categories. Evasion-aware retraining will incorporate adversarially perturbed samples into the training distribution to assess whether robustness can be recovered without sacrificing accuracy on unperturbed traffic.

C. Scalability

Validation was conducted in a single-site virtualized laboratory. Behavior under high-throughput conditions (10 Gbps+), distributed multi-site deployments, or cloud-native architectures was not evaluated. Planned extensions include distributed Zeek cluster deployment and ML inference parallelization. The orchestration engine's inter-process queue architecture is designed to support horizontal scaling: additional inference workers can be added without modification to the threat registry or deduplication logic, as all state is maintained in the central registry process, providing a natural foundation for multi-node deployment.

D. False Positive Characterization in Production

As discussed in Section VI-B, the zero-FP evaluation result reflects a controlled test environment. Longitudinal false-positive monitoring under production-realistic traffic — including concurrent administrative activity, network scans by legitimate tools, and high-volume backup traffic — is the highest-priority prerequisite for operational deployment and is planned as immediate future work.

E. Future Directions

(1) Federated learning for multi-site model updates without centralizing raw traffic data [26],[27]. (2) Evaluation on CICIDS2018 and CIC-DDoS2019 [17] for external benchmark comparability. (3) MITRE ATT&CK framework mapping to classification outputs for standardized threat intelligence reporting [28]. (4) Honeypot integration for extended attacker dwell-time analysis during detection. (5) Integration of SHAP-based explainability [29],[30] to improve analyst trust and interpretability of ML classification decisions.

VIII. CONCLUSION

This paper presented an AI-powered, multi-layer network defense system that simultaneously advances three dimensions where prior systems universally fall short: dataset quality, detection architecture, and mitigation automation. Prior systems stop short of automated remediation [6], rely on generic benchmark data [14],[15], or omit cross-engine alert correlation [16]. The present work addresses all three deficiencies in a single integrated framework.

By generating a custom environment-specific training dataset of 80,000 labeled flow samples — with label quality validated through manual audit — the system avoids the generalization failures of benchmark-trained models. By integrating XGBoost, Suricata, Zeek, and rule-based heuristics under a unified orchestration engine with intelligent deduplication, the system achieves comprehensive

detection coverage while eliminating alert flooding. By coupling detection directly to automated pfSense firewall rule injection, the system closes the detection-to-remediation gap characterizing prior work — the first live-validated implementation of this kind in the reviewed literature.

Experimental evaluation across 240 independent trials demonstrated 100% detection rate and mean MTMM of 872 ms (± 148 ms overall; per-scenario σ range ± 21 –47 ms). Baseline comparison confirmed the multi-layer architecture outperforms all three single-method subsystems across all eight evaluated attack scenarios. The system demonstrates that autonomous real-time network defense is operationally achievable with commodity hardware and open-source tooling — a practical and reproducible contribution to enterprise cybersecurity infrastructure.

Beyond its measured results, this work establishes a reusable architectural blueprint for environment-native IDS development. The open-source toolchain ensures accessibility without proprietary licensing constraints, while the dataset generation pipeline, orchestration engine, and mitigation framework are each independently extensible to future attack taxonomies or alternative classifiers. As adversarial techniques continue to evolve, the multi-layer confirmation model provides inherent resilience — no single layer failure results in missed detection. The present system thus contributes not only a validated defense framework, but a practical foundation for autonomous, adaptive network security.

REFERENCES

- [1] Ericsson, "Ericsson Mobility Report," Ericsson AB, Stockholm, Sweden, Tech. Rep., Nov. 2023. [Online]. Available: <https://www.ericsson.com/en/reports-and-papers/mobility-report> [Accessed: Apr. 12, 2026].
- [2] M. Roesch, "Snort: Lightweight Intrusion Detection for Networks," in Proc. USENIX LISA, Seattle, WA, USA, 1999, pp. 229–238.
- [3] P. Garcia-Teodoro, J. Diaz-Verdejo, G. Macia-Fernandez, and E. Vazquez, "Anomaly-Based Network Intrusion Detection: Techniques, Systems and Challenges," *Comput. Secur.*, vol. 28, no. 1–2, pp. 18–28, 2009.
- [4] IBM Security, "Cost of a Data Breach Report 2023," IBM Corp., Armonk, NY, USA, 2023. [Online]. Available: <https://www.ibm.com/reports/data-breach> [Accessed: Apr. 12, 2026].
- [5] M. Al-Qatf, Y. Lasheng, M. Al-Hababi, and K. Al-Sabahi, "Deep Learning Approach Combining Sparse Autoencoder With SVM for Network Intrusion Detection," *IEEE Access*, vol. 6, pp. 52843–52856, 2018. doi: 10.1109/ACCESS.2018.2869577
- [6] A. Mersni, I. Iliyas, and Q. A. Al-Haija, "A Hybrid Intrusion Detection System for Small and Medium Enterprises," *IEEE Access*, vol. 11, pp. 78234–78251, 2023. doi: 10.1109/ACCESS.2023.3298541
- [7] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, "Survey of Intrusion Detection Systems: Techniques, Datasets and Challenges," *Cybersecurity*, vol. 2, no. 1, p. 20, 2019. doi: 10.1186/s42400-019-0038-7

- [8] S. S. Dhaliwal, A.-A. Nahid, and R. Abbas, "Effective Intrusion Detection System Using XGBoost," *Information*, vol. 9, no. 7, p. 149, 2018. doi: 10.3390/info9070149
- [9] G. B. N. Rao, P. N. Kumar, and M. S. Rao, "An Efficient Intrusion Detection System Using XGBoost for Software-Defined Networks," *Int. J. Intell. Syst. Appl. Eng.*, vol. 11, no. 4, pp. 312–321, 2023. doi: 10.18201/ijisae.2023.348
- [10] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining (KDD)*, San Francisco, CA, USA, 2016, pp. 785–794. doi: 10.1145/2939672.2939785
- [11] R. Vinayakumar, M. Alazab, K. P. Soman, P. Poornachandran, A. Al-Nemrat, and S. Venkatraman, "Deep Learning Approach for Intelligent Intrusion Detection System," *IEEE Access*, vol. 7, pp. 41525–41550, 2019. doi: 10.1109/ACCESS.2019.2895334
- [12] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware Traffic Classification Using Convolutional Neural Network for Representation Learning," in *Proc. IEEE Int. Conf. Inf. Netw. (ICOIN)*, 2017, pp. 712–717. doi: 10.1109/ICOIN.2017.7899588
- [13] M. Alsaidi, M. Al-Sarem, and F. Saeed, "Combining Zeek and Suricata for Improved Network Intrusion Detection," *J. Inf. Secur. Appl.*, vol. 72, p. 103389, 2023. doi: 10.1016/j.jisa.2022.103389
- [14] M. Tavallaei, E. Bagheri, W. Lu, and A. A. Ghorbani, "A Detailed Analysis of the KDD CUP 99 Data Set," in *Proc. IEEE Symp. Computational Intelligence for Security and Defense Applications (CISDA)*, 2009, pp. 1–6. doi: 10.1109/CISDA.2009.5356528
- [15] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," in *Proc. Int. Conf. Inf. Syst. Secur. Privacy (ICISSP)*, Funchal, Portugal, 2018, pp. 108–116. doi: 10.5220/0006639801080116
- [16] T. Pietraszek, "Using Adaptive Alert Classification to Reduce False Positives in Intrusion Detection," in *Proc. Int. Symp. Res. Attacks, Intrusions Defenses (RAID)*, *Lect. Notes Comput. Sci.*, vol. 3224, Berlin, Germany: Springer, 2004, pp. 102–124. doi: 10.1007/978-3-540-30143-1_6
- [17] I. Sharafaldin, A. H. Lashkari, S. Hakak, and A. A. Ghorbani, "Developing Realistic Distributed Denial of Service (DDoS) Attack Dataset and Taxonomy," in *Proc. IEEE DASC/PiCom/CBDCoM/CyberSciTech*, Fukuoka, Japan, 2019, pp. 91–98. doi: 10.1109/DASC/PiCom/CBDCoM/CyberSciTech.2019.00025
- [18] N. Moustafa and J. Slay, "UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems," in *Proc. IEEE MilCIS*, 2015, pp. 1–6. doi: 10.1109/MilCIS.2015.7348942
- [19] Y. Zhou, G. Cheng, S. Jiang, and M. Dai, "Building an Efficient Intrusion Detection System Based on Feature Selection and Ensemble Classifier," *Comput. Netw.*, vol. 174, p. 107247, 2020. doi: 10.1016/j.comnet.2020.107247
- [20] S. Garcia, M. Parmisano, and M. J. Erquiaga, "Zeek-based Machine Learning Behavioral IDS: The Stratosphere Linux IPS," in *Proc. 24th Int. Symp. Res. Attacks, Intrusions Defenses (RAID)*, San Sebastian, Spain, 2021, pp. 152–165. doi: 10.1145/3471621.3471845
- [21] S. Bhatt, P. K. Manadhata, and L. Zomlot, "The Operational Role of Security Information and Event Management Systems," *IEEE Secur. Privacy*, vol. 12, no. 5, pp. 35–41, 2014. doi: 10.1109/MSP.2014.103
- [22] M. H. Muttaqien, M. Niswar, S. Syarif, and Z. Zainuddin, "Enhancing Network Intrusion Detection for TLS Traffic Using Deep Learning," *Eng. Technol. Appl. Sci. Res.*, vol. 15, no. 6, pp. 28530–28536, 2025. doi: 10.48084/etasr.13267
- [23] I. Corona, G. Giacinto, and F. Roli, "Adversarial Attacks Against Intrusion Detection Systems: Taxonomy, Solutions and Open Issues," *Inf. Sci.*, vol. 239, pp. 201–225, 2013. doi: 10.1016/j.ins.2013.03.022
- [24] D. Apruzzese, M. Colajanni, L. Ferretti, A. Guido, and M. Marchetti, "On the Effectiveness of Machine and Deep Learning for Cyber Security," in *Proc. 10th Int. Conf. Cyber Conflict (CyCon)*, 2018, pp. 371–390. doi: 10.23919/CYCON.2018.8405026
- [25] K. He, D. D. Kim, and M. R. Asghar, "Adversarial Machine Learning for Network Intrusion Detection Systems: A Comprehensive Survey," *IEEE Commun. Surv. Tutorials*, vol. 25, no. 1, pp. 538–566, 2023. doi: 10.1109/COMST.2022.3233793
- [26] T. D. Nguyen, S. Marchal, M. Miettinen, H. Fereidooni, N. Asokan, and A.-R. Sadeghi, "DIoT: A Federated Self-Learning Anomaly Detection System for IoT," in *Proc. IEEE 39th Int. Conf. Distributed Computing Systems (ICDCS)*, 2019, pp. 756–767. doi: 10.1109/ICDCS.2019.00080
- [27] M. H. Wardana and P. Sukarno, "Taxonomy and Survey of Collaborative Intrusion Detection System Using Federated Learning," *ACM Comput. Surv.*, vol. 57, no. 4, pp. 1–36, 2024. doi: 10.1145/3698398
- [28] S. Peng, H. Wu, and Q. Chen, "MITRE ATT&CK-Based Evaluation of Network Intrusion Detection Systems: A Comprehensive Review," *IEEE Trans. Netw. Serv. Manag.*, vol. 20, no. 2, pp. 1478–1495, 2023. doi: 10.1109/TNSM.2022.3232349
- [29] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017, pp. 4765–4774. doi: 10.5555/3294996.3295091
- [30] D. Gaspar, P. Silva, and C. Silva, "Explainable AI for Intrusion Detection Systems: LIME and SHAP Applicability on Multi-Layer Perceptron," *IEEE Access*, vol. 12, pp. 30164–30175, 2024. doi: 10.1109/ACCESS.2024.3368377

APPENDIX: PROJECT ARTIFACTS

The following artifacts were produced as part of this graduation project and are available upon request from the supervising institution: (1) TrafficCollector data collection module (Python/Scapy); (2) XGBoost model training and evaluation pipeline (Python/scikit-learn); (3) AI-Powered Network Defense System v2.0 source code; (4) live attack test logs (final_test_of_attacks.txt); (5) network architecture topology diagram (My_Complete_Network.png).